

Bratra Integration Platform — Architecture & Integration Guide

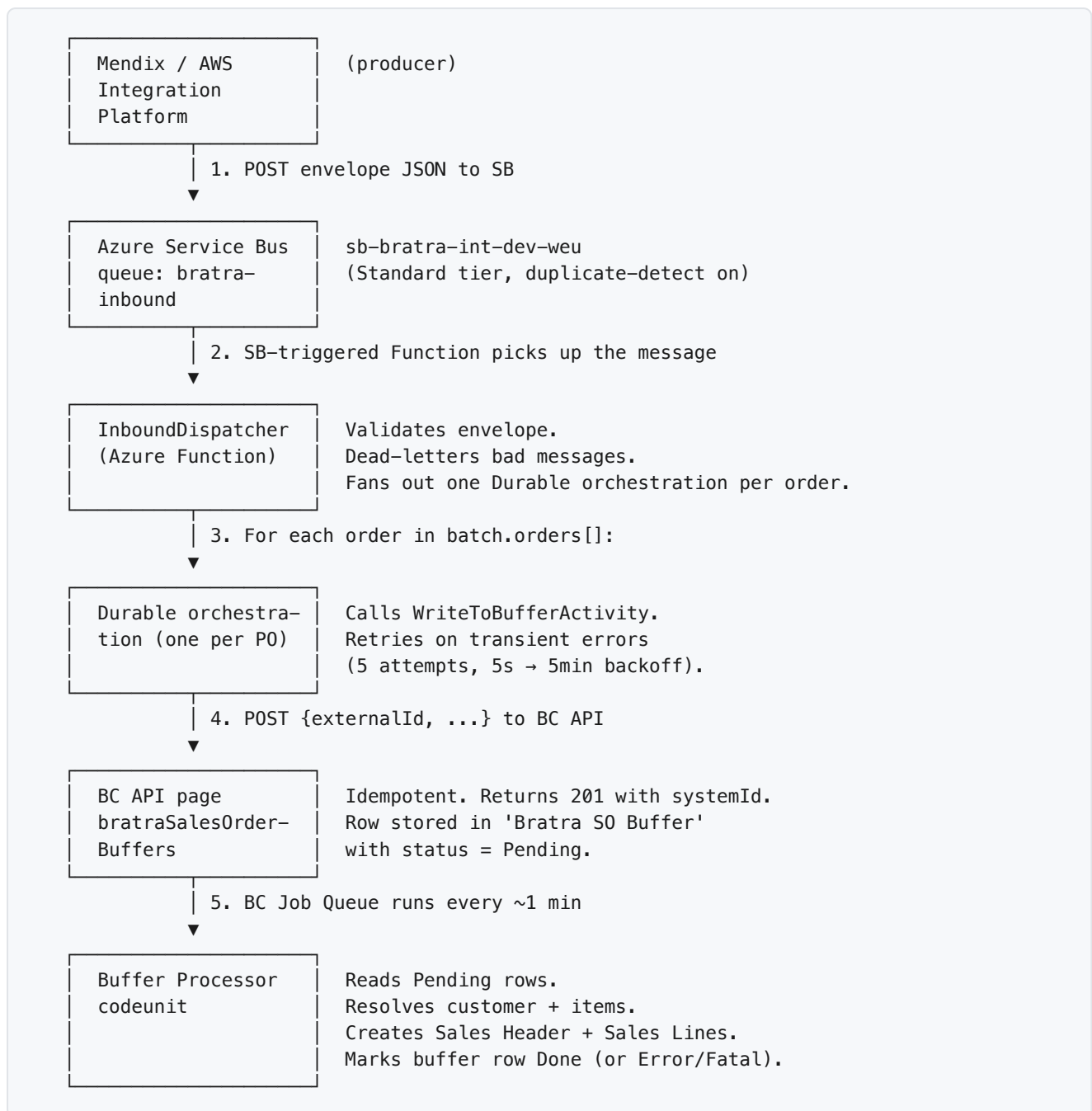
1. Overview

The Bratra Integration Platform is the middleware that connects Bratra's upstream systems (Mendix on AWS) to Microsoft Dynamics 365 Business Central. It receives canonical JSON messages, validates them, hands them off to a per-order workflow, and writes them into a BC buffer table. From there, BC's own Job Queue creates the actual Sales Orders.

Three things to remember:

1. **You push, we pull internally.** Producers send messages to one Azure Service Bus queue. We never call you back. Failures land in the Dead Letter Queue (DLQ), not in a callback.
2. **One queue for everything.** Sales orders today, vendors and items later — all use the same `bratra-inbound` queue. The Function dispatches by `meta.name` internally.
3. **The Integration Platform is leading for transport; BC is leading for processing.** The middleware does shape and routing. Business logic (pricing, posting groups, dimensions) lives in BC.

2. End-to-end data flow



3. Quick start — using the Postman collection

A pre-built Postman pack lives at `docs/postman/` in this repository. Two files:

- `bratra-inbound.postman_collection.json` — four pre-built requests.
- `bratra-inbound.postman_environment.json` — environment template (one secret to fill in).

Setup (one-time, ~5 minutes)

1. **Get the SAS key.** The collection authenticates with a Send-only SAS rule named `producer-send` on the `bratra-inbound` queue. Two ways:

- Ask Leo / the platform team for it via 1Password (preferred).
- Get it yourself in Azure portal: Resource Group `rg-bratra-int-dev-weu` → Service Bus `sb-bratra-int-dev-weu` → Queues → `bratra-inbound` → Shared access policies → `producer-send` → Primary Key.

The key only allows **sending** to this one queue. It cannot read messages or change the queue.

- Import into Postman.** File → Import → drag both JSON files. Top-right environment dropdown → select **Bratra Inbound (dev)**. Click the eye icon → Edit → paste the key into `sb_key_value` → Save.
- Send your first message.** Click request **1. Send valid Action order envelope (happy path)** → Send. Expect HTTP 201 from Service Bus (body empty — that's normal). Within ~1 second a new row appears in BC's `Bratra Sales Order Buffers` page with status `Pending`.

The four requests

#	Request	Expected result	Use it for
1	Send valid Action order envelope	SB 201, BC row appears, status Pending → Done within ~1 min	Happy path smoke test
2	Send custom envelope	Same wire, blank body	Your own scenarios
3	Send malformed envelope (DLQ demo)	SB 201, message dead-lettered with reason <code>UnknownMessageName</code>	Confirm DLQ behaviour without polluting BC
4	Send multi-order batch	SB 201, three orchestrations fan out, three rows in BC	Verify batch processing — PO 4002845710/4002845711/4002845712 are visually distinct

What the pre-request script does

Each request runs JavaScript before sending that:

- **Generates a fresh SAS token** (HMAC-SHA256, 10-minute TTL). You don't need to refresh it manually.
- **Generates a fresh `meta.messageId`** (UUIDv4) per send, so duplicate detection treats each click as a new logical message.
- **Generates a fresh `correlationId`** prefixed with `wesley-{timestamp}` so it's easy to find in our logs.

Don't edit the pre-request script unless you know what you're doing — re-importing the collection replaces it.

Verifying the message landed in BC

After SB returns 201, open BC → search **Bratra Integration** → **Sales Order Buffers** page. The newest row will have:

- **External ID** = `BRA-AC-{messageId}-{poNumber}` (e.g. `BRA-AC-abc12345-...-4002845709`)
- **Status** = `Pending` initially
- Within ~1 minute, BC Job Queue moves it through `Processing` → `Done` (success) or `Error` / `Fatal` (something didn't match BC master data).
- The 6 logistics fields (Contract Number, Carrier Code/Name, Distribution Center Code/Name, Unloading Location) populated from your envelope.
- The line **Item No.** set to your `articleNumberSupplier` value (EAN).

You can also query the row via the BC API directly:

```
GET .../bratraSalesOrderBuffers?$filter=externalId eq 'BRA-AC-...'
```

4. The message contract

4.1 Envelope

Every message is a JSON envelope wrapping the domain payload. Schemas:

- `schemas/message-envelope-v1.json` — the envelope.
- `schemas/action-order-v1.json` — Action order payload (today's only domain).

```
{
  "meta": {
    "messageId": "<UUIDv4 or ULID, unique per message>",
    "correlationId": "<business-level id, stable across retries>",
    "name": "ActionOrderBatchV1",
    "version": "1",
    "source": "BratraIntegrationPlatform",
    "legalEntity": "BRATRA-NL",
    "occurredOnUtc": "2026-05-07T14:23:45.678Z"
  },
  "payload": {
    "orders": [ /* see action-order-v1.json - N orders per envelope */ ]
  }
}
```

4.2 Action order payload

One envelope can carry N orders. Each order has its own `poNumber` , header fields, and lines.

```

{
  "poNumber": "4002845709",
  "orderType": "Regular order",
  "contractNumber": "A250002419",
  "carrier": { "code": "50015", "name": "ID Freight Netherlands B.V." },
  "distributionCenter": { "code": "8990", "name": "Wallersdorf",
    "unloadingLocation": "" },
  "dates": {
    "reqDeliveryDate": "2026-05-15", "expDeliveryDate": "2026-05-15",
    "reqETD": null, "expETD": null, "eta": null
  },
  "shipping": { /* carrier metadata; not surfaced in BC */ },
  "lines": [
    {
      "lineNumber": 10,
      "articleNumberAction": "3007781",
      "articleNumberSupplier": "8721008420981",
      "articleDescription": "hr t-shirt mcg katoen m-xxl wit div.var",
      "category": "11", "logisticGroup": "009", "allocation": null,
      "quantities": { "reqQuantity": 2352, "expQuantity": 2352, "palletPattern": 1176,
        "pallets": 2 },
      "pricing": { "unitPrice": 2.35, "currency": "EUR" },
      "compliance": { "hg": "", "adr": null, "icpe": 1510 }
    }
  ]
}

```

A full concrete example is shipped at `schemas/samples/action-order-envelope-example.json` — verbatim valid input for the dev environment.

4.3 The 6 logistics fields

As of 2026-05-07 the BC buffer captures six logistics fields explicitly (rather than only forensically). Every send is expected to populate them when present. They are visible on the Bratra SO Buffer card in BC.

Wire path	BC field	Notes
<code>contractNumber</code>	Contract Number	Bratra raamcontract identifier
<code>carrier.code</code>	Carrier Code	Carrier identifier (e.g. 50015)
<code>carrier.name</code>	Carrier Name	Free-text
<code>distributionCenter.code</code>	Distribution Center Code	DC identifier (e.g. 8990)
<code>distributionCenter.name</code>	Distribution Center Name	DC display name (e.g. Wallersdorf)
<code>distributionCenter.unloadingLocation</code>	Unloading Location	Optional dock / position

4.4 Service Bus message properties

When you send via the Azure SDK, set the following properties on the `ServiceBusMessage`. The Postman collection sets the equivalent `BrokerProperties` header.

Property	Value	Why
Body	UTF-8 bytes of the JSON envelope	The payload
ContentType	<code>application/json</code>	Enables fast deserialisation
MessageId	equal to <code>meta.messageId</code>	SB-native duplicate detection
CorrelationId	equal to <code>meta.correlationId</code>	Flows through logs and response routing
Subject	equal to <code>meta.name</code> (e.g. <code>ActionOrderBatchV1</code>)	Routing without parsing body
<code>ApplicationProperties["schemaVersion"]</code>	equal to <code>meta.version</code>	Per-version routing rules
<code>ApplicationProperties["source"]</code>	equal to <code>meta.source</code>	Audit + routing

C# example

```
var envelopeJson = JsonSerializer.Serialize(envelope, jsonOptions);

var message = new ServiceBusMessage(envelopeJson)
{
    ContentType = "application/json",
    MessageId = envelope.Meta.MessageId,
    CorrelationId = envelope.Meta.CorrelationId,
    Subject = envelope.Meta.Name,
    ApplicationProperties =
    {
        ["schemaVersion"] = envelope.Meta.Version,
        ["source"] = envelope.Meta.Source,
    }
};

await sender.SendMessageAsync(message);
```

5. Connection details

Parameter	Dev value
Service Bus namespace	sb-bratra-int-dev-weu
FQDN	sb-bratra-int-dev-weu.servicebus.windows.net
Inbound queue	bratra-inbound
Dead Letter Queue	bratra-inbound/\$DeadLetterQueue (sibling)
Auth (dev)	SAS Send-only key producer-send (1Password)
Auth (prd, planned)	Managed Identity or federated SP
Content-Type	application/json; charset=utf-8
Max message size	256 KiB (Standard tier) — split large batches

Note: dev and prd will run on separate Service Bus namespaces (sb-bratra-int-prd-weu once provisioned).

6. BC architecture — what happens to your message

Once the Function POSTs your order to BC's bratraSalesOrderBuffers API, BC takes over. Here is what runs on the BC side, owned by ERP Company.

6.1 Buffer tables

- **Bratra S0 Buffer BIN** (table 50100) — one row per order. Stores all header fields (the 6 logistics fields, customer info, dates, posting description). Each row has a status enum: Pending , Processing , Done , Error , Fatal , Cancelled .
- **Bratra S0 Buffer Line BIN** — child rows; one per lines[i] element from the wire.
- **Bratra Activity Log BIN** — append-only audit log per row.

6.2 The API page

bratraSalesOrderBuffers is the OData v2.0 API page our Function POSTs to.

- **Idempotent.** externalId is the natural key. Re-POSTing the same externalId returns HTTP 400 with code Internal_RecordError ; the Function treats this as success (the message was already accepted).
- **Deep insert.** One POST creates header + child lines atomically.
- **Returns** systemId , entryNo , status .

6.3 Buffer Processor codeunit

A codeunit (BratraBufferProcessorBIN) runs every minute via BC's Job Queue. Per tick:

1. Reaps stale `Processing` rows (rows stuck for too long).
2. Picks up `Pending` and `Error` rows.
3. Processes each row in its own transaction (`Codeunit.Run` for isolation).
4. Creates the actual BC Sales Header + Sales Lines, copying the buffer columns into the standard Sales document.
5. On success: marks the buffer `Done` , fills `Created Document No.` .
6. On transient failure: marks `Error` , increments retry count, schedules a retry with exponential backoff (base $\times 2^{(n-1)}$, capped at 24h).
7. On permanent failure: marks `Fatal` , fills `Error Message` . User-visible — needs operator attention.

6.4 Error classification

Errors that bubble up from `Sales-Header.Validate` etc. are routed through `BratraErrorClassifierBIN` . The classifier decides whether an error is transient (retry) or fatal (stop and alert). Examples:

- "Item X does not exist" → Fatal.
- "Network glitch / lock timeout" → Transient.
- "BC service throttled (429)" → Transient.

This avoids hammering BC with retries on errors that won't get better.

6.5 Customer and item resolution

These are explicit design decisions, captured here so producers know which fields BC reads.

- **Customer.** The Function sends `customerNo` from an app-config map (e.g. `CUSTOMER_MAP__action__BRATRA-NL=C00006`). If empty, BC's `OnResolveCustomerByExternalId` event fires. If still unresolved, BC's `Default Customer No.` from `Bratra Integration Setup` is used. All three failing → `Fatal` .
- **Item.** Currently the Function sets `salesOrderBufferLines[i].no` equal to `articleNumberSupplier` (the EAN). This works as long as BC items are seeded with `No.` = EAN. The long-term plan is to wire BC's `OnResolveItemByExternalId` event so the Function only sends `externalItemId` and BC resolves to the proper `Item No.` .

6.6 Permissions

The Function authenticates as the SP `Bratra-Integration-BC-Caller` (Bratra Entra tenant). In BC it is granted the `BRATRA INT BIN` permission set on the target company via the AAD Applications page (9089).

6.7 Telemetry

Both sides emit App Insights events with the same `correlationId` . BC's event IDs `BRA-IN-0001` through `BRA-IN-0007` cover ingest, validation, processing, retry, fatal. Custom dimensions include `domain` , `bufferEntryNo` , `externalId` , `correlationId` , `status` , `retryCount` . This lets ops trace

a single message from SB → Function → buffer → Sales Order in one App Insights query.

7. Idempotency

The Integration Platform deduplicates on `meta.messageId` . If you retry the **same logical batch**, reuse the **same `messageId`** . SB's duplicate-detect window suppresses the retry at ingest.

Different logical batches must use different `messageId` s. Always.

`correlationId` is yours — a natural business key like `action-batch-2026-05-07-001` is fine. Keep it stable across retries; change it across genuinely new batches.

On the BC side, the `externalId` (`BRA-AC-{messageId}-{poNumber}`) provides a second idempotency check: if the same `externalId` is POSTed twice, BC returns 400 and the Function treats it as success.

8. Errors and the Dead Letter Queue

No success messages are sent back to producers. Failures land on Service Bus's Dead Letter Queue.

8.1 Failure classes

Failure class	Caused by	Dead-letter reason
Bad envelope JSON	Malformed JSON	<code>InvalidEnvelopeJson</code>
Missing envelope.meta	<code>meta</code> is null or absent	<code>EnvelopeMetaMissing</code>
Unknown message name	<code>meta.name</code> not in dispatcher registry	<code>UnknownMessageName</code>
Payload schema violation	Payload doesn't match the registered schema	<code>PayloadSchemaViolation</code>
Empty payload	<code>payload.orders</code> empty or missing	<code>EmptyPayload</code>
BC fatal rejection	BC rejects with a non-retryable error after retries	<code>BcFatalRejection</code>
Max delivery count exceeded	All transient retries exhausted	<code>MaxDeliveryCountExceeded</code>

8.2 What a DLQ message looks like

The dead-lettered message retains the original envelope, `MessageId` , `CorrelationId` , and `ApplicationProperties` from the producer. The Function adds:

- `DeadLetterReason` — short code (one of the values above).
- `DeadLetterErrorDescription` — the full detail (often the BC error verbatim, truncated to 4000 chars).

- `ApplicationProperties["errorSource"]` — `dispatcher` | `orchestrator` | `bcActivity` .
- `ApplicationProperties["occurredOnUtc"]` — when it failed.

8.3 How to inspect the DLQ

- Azure portal: `bratra-inbound` queue → Service Bus Explorer → Dead-letter tab → peek.
- Or in code, receive from `bratra-inbound/$DeadLetterQueue` .

Try Postman request #3 once and you'll see exactly what a dead-lettered entry looks like.

8.4 Monitoring

App Insights alerts on DLQ depth `> 0` for more than 5 minutes. The alert routes to Variables and ERP Company. If no one contacts you about a batch, it processed.

8.5 Re-sending

Resubmit the corrected envelope with a **new** `messageId` . Reuse the original `correlationId` so the two attempts are tied together in logs.

9. Validation

Before pushing to dev, validate locally against the JSON schemas:

```
npx -p ajv-cli -p ajv-formats ajv validate \  
  --spec=draft2020 -c ajv-formats \  
  -s schemas/message-envelope-v1.json \  
  -s schemas/action-order-v1.json \  
  -d schemas/samples/action-order-envelope-example.json
```

Schema validation failures on our side are **non-retryable** and go straight to DLQ.

10. Reference

Resource	Path
Envelope schema	<code>schemas/message-envelope-v1.json</code>
Action order schema	<code>schemas/action-order-v1.json</code>
Example envelope	<code>schemas/samples/action-order-envelope-example.json</code>
Postman collection	<code>docs/postman/bratra-inbound.postman_collection.json</code>
Postman environment	<code>docs/postman/bratra-inbound.postman_environment.json</code>
BC API contract	<code>~/BusinessCentral/Bratra.Integration/docs/api-contract.md</code>
BC operator manual	<code>~/BusinessCentral/Bratra.Integration/docs/manual.md</code>
Function source	<code>src/Bratra.Integration.Functions/</code>
Wire→BC mapping	<code>docs/bc-mapping.md</code>

Last updated 2026-05-11. For changes or corrections, contact Leo Sharma.